

Lab 6 Name-Chirag Miglani

Roll no. 1/23/SET/BCS/414

Email Spam or not spam with 62 and 32 neurons

```
import pandas as pd
import nltk
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download necessary NLTK data
nltk.download('stopwords')
nltk.download('punkt')

# Load your dataset (update the filename to match yours)
df = pd.read_csv('spam_or_not_spam.csv')

# Quick look at the data
print(df.head())
```

```

              email  label
0  date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...      0
1  martin a posted tassos papadopoulos the greek ...      0
2  man threatens explosion in moscow thursday aug...      0
3  klez the virus that won t die already the most...      0
4  in adding cream to spaghetti carbonara which ...      0
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder

# Download necessary NLTK datasets
nltk.download('stopwords')
nltk.download('punkt')

# Load your dataset - Replace 'your_file.csv' with your actual filename
df = pd.read_csv('spam_or_not_spam.csv')

# Drop any rows where the email content is missing
df.dropna(subset=['email'], inplace=True)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True
```

```
stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def clean_email(text):
    # 1. Remove URLs, email addresses, and non-alphabetic characters
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove links
```

```

text = re.sub(r'\S+@\S+', '', text) # Remove actual email addresses
text = re.sub(r'[^a-zA-Z]', ' ', text) # Remove numbers and punctuation

# 2. Lowercase and Tokenize
text = text.lower()
words = nltk.word_tokenize(text)

# 3. Remove Stop words and Stemming
# We keep words like 'free' or 'win' but remove 'the', 'is', etc.
cleaned = [stemmer.stem(w) for w in words if w not in stop_words]

return " ".join(cleaned)

# Apply cleaning to your 'email' column
df['cleaned_email'] = df['email'].apply(clean_email)

```

```

# 1. Encode the Labels (Ham = 0, Spam = 1)
# Assuming your label column is named 'label' or 'target'
le = LabelEncoder()
df['label_encoded'] = le.fit_transform(df['label'])

# 2. TF-IDF Vectorization
# We limit to 3000 features to keep the model efficient
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['cleaned_email']).toarray()
y = df['label_encoded'].values

print("Preprocessing Complete!")
print(f"Dataset Shape: {X.shape}") # Shows (number of emails, number of unique words)

```

```

Preprocessing Complete!
Dataset Shape: (2999, 3000)

```

```

import numpy as np

# 1. Initialize Weights and Bias
# X.shape[1] is the number of features (3000 if you followed the previous step)
weights = np.zeros(X.shape[1])
bias = 0.0
learning_rate = 0.01
epochs = 10 # Number of times to loop through the entire dataset

# 2. Training Loop
for epoch in range(epochs):
    total_error = 0
    for i in range(len(X)):
        # --- FORWARD PASS ---
        # Linear combination
        z = np.dot(X[i], weights) + bias
        # Activation (Step Function: 1 if z >= 0 else 0)
        y_pred = 1 if z >= 0 else 0

        # --- CALCULATE LOSS / ERROR ---
        error = y[i] - y_pred
        total_error += abs(error)

        # --- BACKWARD PASS (LEARN WEIGHTS + BIAS) ---
        if error != 0:
            weights += learning_rate * error * X[i]
            bias += learning_rate * error

    print(f"Epoch {epoch+1}/{epochs} - Errors: {total_error}")

print("\nTraining Complete!")

```

```

Epoch 1/10 - Errors: 3
Epoch 2/10 - Errors: 4
Epoch 3/10 - Errors: 4
Epoch 4/10 - Errors: 4
Epoch 5/10 - Errors: 4
Epoch 6/10 - Errors: 4
Epoch 7/10 - Errors: 4
Epoch 8/10 - Errors: 4
Epoch 9/10 - Errors: 6
Epoch 10/10 - Errors: 6

```

```

Training Complete!

```

```
from sklearn.model_selection import train_test_split

# Split the data (80% for training, 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")
```

```
Training set size: 2399
Testing set size: 600
```

```
import numpy as np

# Initialize weights for the training set features
weights = np.zeros(X_train.shape[1])
bias = 0.0
learning_rate = 0.05
epochs = 25

# Training Loop using ONLY X_train
for epoch in range(epochs):
    total_error = 0
    for i in range(len(X_train)):
        # Forward Pass
        z = np.dot(X_train[i], weights) + bias
        y_pred_temp = 1 if z >= 0 else 0

        # Calculate Loss (Error)
        error = y_train[i] - y_pred_temp
        total_error += abs(error)

    # Backward Pass (Update Weights and Bias)
    if error != 0:
        weights += learning_rate * error * X_train[i]
        bias += learning_rate * error

    print(f"Epoch {epoch+1}/{epochs} - Errors: {total_error}")
```

```
Epoch 1/25 - Errors: 167
Epoch 2/25 - Errors: 48
Epoch 3/25 - Errors: 22
Epoch 4/25 - Errors: 20
Epoch 5/25 - Errors: 16
Epoch 6/25 - Errors: 7
Epoch 7/25 - Errors: 6
Epoch 8/25 - Errors: 4
Epoch 9/25 - Errors: 2
Epoch 10/25 - Errors: 0
Epoch 11/25 - Errors: 0
Epoch 12/25 - Errors: 0
Epoch 13/25 - Errors: 0
Epoch 14/25 - Errors: 0
Epoch 15/25 - Errors: 0
Epoch 16/25 - Errors: 0
Epoch 17/25 - Errors: 0
Epoch 18/25 - Errors: 0
Epoch 19/25 - Errors: 0
Epoch 20/25 - Errors: 0
Epoch 21/25 - Errors: 0
Epoch 22/25 - Errors: 0
Epoch 23/25 - Errors: 0
Epoch 24/25 - Errors: 0
Epoch 25/25 - Errors: 0
```

```
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Use the weights and bias to predict on the UNSEEN X_test
z_test = np.dot(X_test, weights) + bias
y_pred = [1 if val >= 0 else 0 for val in z_test]

# 2. Results
print(f"Final Test Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

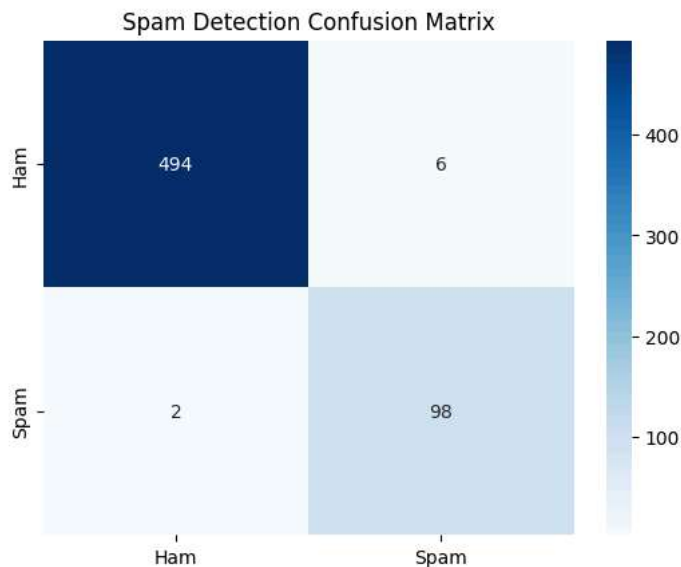
# 3. Confusion Matrix Visualization
cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Ham', 'Spam'], yticklabels=['Ham', 'Spam'])
plt.title('Spam Detection Confusion Matrix')
plt.show()
```

Final Test Accuracy: 98.67%

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.99	0.99	500
1	0.94	0.98	0.96	100
accuracy			0.99	600
macro avg	0.97	0.98	0.98	600
weighted avg	0.99	0.99	0.99	600



```
import numpy as np
import matplotlib.pyplot as plt

# Initialize variables
weights = np.zeros(X_train.shape[1])
bias = 0.0
learning_rate = 0.01
epochs = 20 # Increased epochs to see a better curve
loss_history = []
accuracy_history = []

for epoch in range(epochs):
    total_error = 0
    correct_predictions = 0

    for i in range(len(X_train)):
        # Forward Pass
        z = np.dot(X_train[i], weights) + bias
        y_pred_temp = 1 if z >= 0 else 0

        # Track accuracy
        if y_pred_temp == y_train[i]:
            correct_predictions += 1

        # Calculate Loss (Error)
        error = y_train[i] - y_pred_temp
        total_error += abs(error)

        # Backward Pass (Update weights/bias)
        if error != 0:
            weights += learning_rate * error * X_train[i]
            bias += learning_rate * error

    # Record metrics for plotting
    loss_history.append(total_error)
    accuracy_history.append(correct_predictions / len(X_train))
```

```
print(f"Epoch {epoch+1} done.")
```

```
Epoch 1 done.
Epoch 2 done.
Epoch 3 done.
Epoch 4 done.
Epoch 5 done.
Epoch 6 done.
Epoch 7 done.
Epoch 8 done.
Epoch 9 done.
Epoch 10 done.
Epoch 11 done.
Epoch 12 done.
Epoch 13 done.
Epoch 14 done.
Epoch 15 done.
Epoch 16 done.
Epoch 17 done.
Epoch 18 done.
Epoch 19 done.
Epoch 20 done.
```

```
plt.figure(figsize=(12, 5))
```

```
# Plotting Loss Curve
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(range(1, epochs + 1), loss_history, color='red', marker='o')
```

```
plt.title('Loss Curve (Total Errors per Epoch)')
```

```
plt.xlabel('Epochs')
```

```
plt.ylabel('Total Error Count')
```

```
plt.grid(True)
```

```
# Plotting Accuracy Curve
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(range(1, epochs + 1), accuracy_history, color='green', marker='o')
```

```
plt.title('Accuracy Curve over Epochs')
```

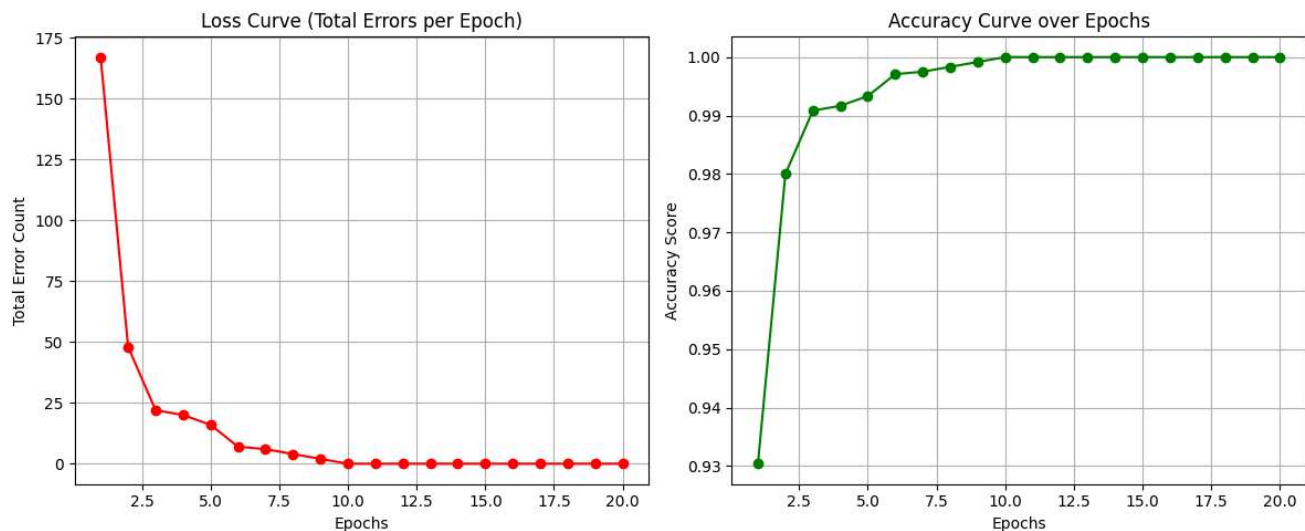
```
plt.xlabel('Epochs')
```

```
plt.ylabel('Accuracy Score')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
import pickle
```

```
# Save the weights, bias, and the tfidf vectorizer
```

```
model_data = {
```

```
    'weights': weights,
```

```
    'bias': bias,
```

```

    'tfidf': tfidf
}

with open('spam_perceptron_model.pkl', 'wb') as f:
    pickle.dump(model_data, f)

from google.colab import files
files.download('spam_perceptron_model.pkl')

```

```

import numpy as np

# --- 1. Helper Functions ---
def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# --- 2. Initialization ---
input_size = X_train.shape[1]
hidden1_size = 64
hidden2_size = 32
output_size = 1

# Weights and Biases
np.random.seed(42) # For consistent results
W1 = np.random.randn(input_size, hidden1_size) * 0.01
b1 = np.zeros((1, hidden1_size))

W2 = np.random.randn(hidden1_size, hidden2_size) * 0.01
b2 = np.zeros((1, hidden2_size))

W3 = np.random.randn(hidden2_size, output_size) * 0.01
b3 = np.zeros((1, output_size))

# --- 3. Training Loop ---
learning_rate = 0.1
epochs = 50
loss_history = []

for epoch in range(epochs):
    # Forward Pass
    # Layer 1
    z1 = np.dot(X_train, W1) + b1
    a1 = relu(z1)

    # Layer 2
    z2 = np.dot(a1, W2) + b2
    a2 = relu(z2)

    # Output Layer
    z3 = np.dot(a2, W3) + b3
    a3 = sigmoid(z3)

    # Calculate Binary Cross-Entropy Loss
    m = y_train.shape[0]
    y_train_reshaped = y_train.reshape(-1, 1)
    loss = -np.mean(y_train_reshaped * np.log(a3 + 1e-8) + (1 - y_train_reshaped) * np.log(1 - a3 + 1e-8))
    loss_history.append(loss)

# --- 4. BACKPROPAGATION ---
# Output Layer Error
dz3 = a3 - y_train_reshaped
dW3 = np.dot(a2.T, dz3) / m
db3 = np.sum(dz3, axis=0, keepdims=True) / m

# Layer 2 Error
da2 = np.dot(dz3, W3.T)
dz2 = da2 * relu_derivative(a2)

```

```

dw2 = np.dot(a1.T, dz2) / m
db2 = np.sum(dz2, axis=0, keepdims=True) / m

# Layer 1 Error
da1 = np.dot(dz2, W2.T)
dz1 = da1 * relu_derivative(a1)
dW1 = np.dot(X_train.T, dz1) / m
db1 = np.sum(dz1, axis=0, keepdims=True) / m

# Update Weights and Biases
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W3 -= learning_rate * dW3
b3 -= learning_rate * db3

if epoch % 10 == 0:
    print(f"Epoch {epoch}, Loss: {loss:.4f}")

print("Training Complete!")

```

```

Epoch 0, Loss: 0.6932
Epoch 10, Loss: 0.6045
Epoch 20, Loss: 0.5506
Epoch 30, Loss: 0.5169
Epoch 40, Loss: 0.4955
Training Complete!

```

```

from sklearn.model_selection import train_test_split

# Splitting data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Reshape y to be a column vector for matrix multiplication
y_train = y_train.reshape(-1, 1)
y_test = y_test.reshape(-1, 1)

```

```

import numpy as np

input_dim = X_train.shape[1]
hidden1 = 64
hidden2 = 32
output = 1

# He/Xavier-style initialization for weights
np.random.seed(42)
W1 = np.random.randn(input_dim, hidden1) * 0.01
b1 = np.zeros((1, hidden1))

W2 = np.random.randn(hidden1, hidden2) * 0.01
b2 = np.zeros((1, hidden2))

W3 = np.random.randn(hidden2, output) * 0.01
b3 = np.zeros((1, output))

```

```

# Activation Functions
def relu(x): return np.maximum(0, x)
def relu_deriv(x): return (x > 0).astype(float)
def sigmoid(x): return 1 / (1 + np.exp(-x))

# Hyperparameters
learning_rate = 0.01
epochs = 200
loss_history = []
accuracy_history = []

for epoch in range(epochs):
    # --- FORWARD PASS ---
    z1 = np.dot(X_train, W1) + b1
    a1 = relu(z1)

    z2 = np.dot(a1, W2) + b2
    a2 = relu(z2)

    z3 = np.dot(a2, W3) + b3

```

```

a3 = sigmoid(z3) # Output prediction

# --- CALCULATE LOSS (BCE) ---
m = y_train.shape[0]
loss = -np.mean(y_train * np.log(a3 + 1e-8) + (1 - y_train) * np.log(1 - a3 + 1e-8))
loss_history.append(loss)

# Track Accuracy
preds = (a3 > 0.5).astype(int)
accuracy_history.append(np.mean(preds == y_train))

# --- BACKWARD PASS (GRADIENT DESCENT) ---
dz3 = a3 - y_train
dW3 = np.dot(a2.T, dz3) / m
db3 = np.sum(dz3, axis=0, keepdims=True) / m

da2 = np.dot(dz3, W3.T)
dz2 = da2 * relu_deriv(a2)
dW2 = np.dot(a1.T, dz2) / m
db2 = np.sum(dz2, axis=0, keepdims=True) / m

da1 = np.dot(dz2, W2.T)
dz1 = da1 * relu_deriv(a1)
dW1 = np.dot(X_train.T, dz1) / m
db1 = np.sum(dz1, axis=0, keepdims=True) / m

# UPDATE WEIGHTS
W3 -= learning_rate * dW3
b3 -= learning_rate * db3
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W1 -= learning_rate * dW1
b1 -= learning_rate * db1

if epoch % 20 == 0:
    print(f"Epoch {epoch}: Loss {loss:.4f}")

```

```

Epoch 0: Loss 0.5515
Epoch 20: Loss 0.5435
Epoch 40: Loss 0.5362
Epoch 60: Loss 0.5295
Epoch 80: Loss 0.5234
Epoch 100: Loss 0.5178
Epoch 120: Loss 0.5127
Epoch 140: Loss 0.5080
Epoch 160: Loss 0.5037
Epoch 180: Loss 0.4998

```

```

import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns

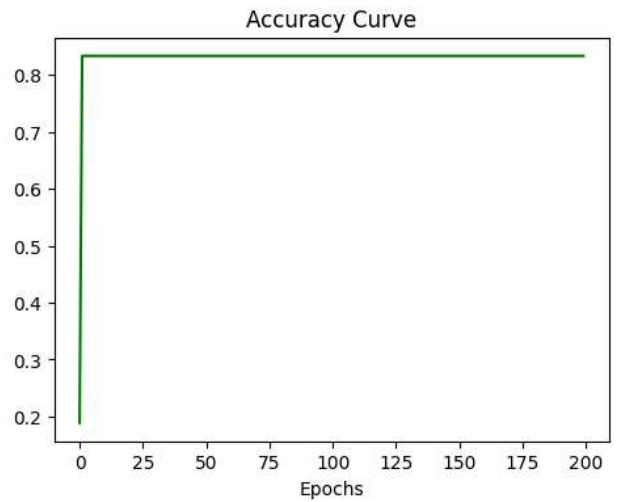
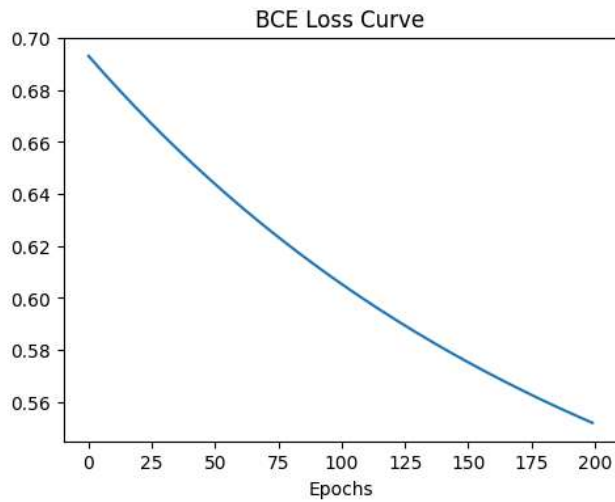
# Plotting Curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(loss_history); plt.title("BCE Loss Curve"); plt.xlabel("Epochs")
plt.subplot(1, 2, 2)
plt.plot(accuracy_history, color='green'); plt.title("Accuracy Curve"); plt.xlabel("Epochs")
plt.show()

# Testing on Unseen Data
z1_test = np.dot(X_test, W1) + b1
a1_test = relu(z1_test)
z2_test = np.dot(a1_test, W2) + b2
a2_test = relu(z2_test)
z3_test = np.dot(a2_test, W3) + b3
final_preds = (sigmoid(z3_test) > 0.5).astype(int)

# Metrics
print("\nAccuracy Score:", accuracy_score(y_test, final_preds))
print("\nClassification Report:\n", classification_report(y_test, final_preds))

# Confusion Matrix
cm = confusion_matrix(y_test, final_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', xticklabels=['Ham', 'Spam'], yticklabels=['Ham', 'Spam'])
plt.title("Confusion Matrix")
plt.show()

```

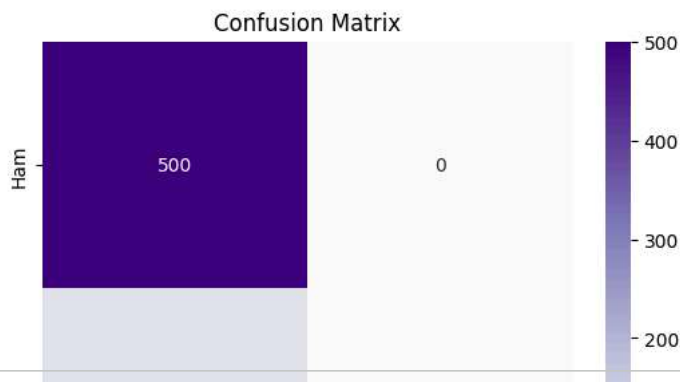



Accuracy Score: 0.8333333333333333

Classification Report:

	precision	recall	f1-score	support
0	0.83	1.00	0.91	500
1	0.00	0.00	0.00	100
accuracy			0.83	600
macro avg	0.42	0.50	0.45	600
weighted avg	0.69	0.83	0.76	600

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



```
import numpy as np
import matplotlib.pyplot as plt

# Initialize lists to store accuracy values
train_accuracy_history = []
test_accuracy_history = []

for epoch in range(epochs):
    # --- TRAINING FORWARD PASS ---
    z1 = np.dot(X_train, W1) + b1
    a1 = relu(z1)
    z2 = np.dot(a1, W2) + b2
    a2 = relu(z2)
    z3 = np.dot(a2, W3) + b3
    a3 = sigmoid(z3)

    # Calculate Training Accuracy
    train_preds = (a3 > 0.5).astype(int)
    train_acc = np.mean(train_preds == y_train)
    train_accuracy_history.append(train_acc)

    # --- TESTING FORWARD PASS ---
    z1_t = np.dot(X_test, W1) + b1
```

```

a1_t = relu(z1_t)
z2_t = np.dot(a1_t, W2) + b2
a2_t = relu(z2_t)
z3_t = np.dot(a2_t, W3) + b3
a3_t = sigmoid(z3_t)

# Calculate Testing Accuracy
test_preds = (a3_t > 0.5).astype(int)
test_acc = np.mean(test_preds == y_test)
test_accuracy_history.append(test_acc)

# --- BACKWARD PASS & WEIGHT UPDATES ---
# (Use your existing backpropagation code here)
# ...

```

```

from sklearn.metrics import classification_report, confusion_matrix

```

```

# Final test predictions using the last weights
z1_f = np.dot(X_test, W1) + b1
a1_f = relu(z1_f)
z2_f = np.dot(a1_f, W2) + b2
a2_f = relu(z2_f)
z3_f = np.dot(a2_f, W3) + b3
final_test_preds = (sigmoid(z3_f) > 0.5).astype(int)

print("\n--- Final Model Metrics ---")
print(classification_report(y_test, final_test_preds, target_names=['Ham', 'Spam']))

```

```

--- Final Model Metrics ---

```

	precision	recall	f1-score	support
Ham	0.83	1.00	0.91	500
Spam	0.00	0.00	0.00	100
accuracy			0.83	600
macro avg	0.42	0.50	0.45	600
weighted avg	0.69	0.83	0.76	600

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 0: no non-zero elements found
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 1: no non-zero elements found
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for class 1: no non-zero elements found
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Start coding or [generate](#) with AI.